

DESARROLLO BACKEND III (PBY2203)

Catalina Portilla R.

Índice

Título(Arial 22 negrita) .....**¡Error! Marcador no definido.**

Subtítulo (Arial 16 negrita).....**¡Error! Marcador no definido.**

Resumen ejecutivo

El presente informe documenta el desarrollo de la solución Backend para la modernización del Banco XYZ, una institución financiera con más de 30 años de operación cuyo sistema de gestión bancaria funcionaba sobre una plataforma legacy basada en COBOL y scripts Shell ejecutados en servidores mainframe. El proyecto abordó tres frentes complementarios: la migración de los procesos batch legacy a Spring Batch, la implementación del patrón Backend for Frontend (BFF) para independizar los tres canales de atención (Web, Móvil y Cajero Automático), y el desarrollo de microservicios de dominio resilientes y seguros orquestados mediante Spring Cloud. La solución resultante, denominada banco-xyz-eft, está compuesta por diez módulos desplegables de forma independiente: un servidor de configuración centralizada (Config Server), un servidor de descubrimiento de servicios (Eureka), un servidor de autorización OAuth2.0, tres microservicios de dominio (Clientes, Cuentas y Pagos), tres BFF especializados por canal, y un servicio de procesos batch (Spring Batch) que reemplaza los scripts legacy. Toda la plataforma es contenedorizada con Docker y preparada para escalar horizontalmente y desplegarse en AWS. Alcance: Migración completa de 3 procesos batch críticos, implementación de 3 BFF por canal y 3 microservicios de dominio con comunicación síncrona (REST vía Eureka) y asíncrona (Kafka), protegidos con OAuth2.0 y con manejo de resiliencia mediante Resilience4j.

Los 5 procesos clave de migración identificados

A partir del análisis del sistema legacy del Banco XYZ, se identificaron los siguientes cinco procesos críticos como objetivo prioritario de migración, por su impacto directo en la operación diaria, el cumplimiento normativo y la experiencia de los clientes:

1. **Reporte de transactiones diarias:** Procesa la totalidad de los movimientos financieros del día (archive movimientos_financieros_diarios.csv) para detector anomalías y generar un resumen diario consolidado.
2. **Cálculo de intereses mensuales:** Aplica tasas diferenciadas sobre cuentas de Ahorro, préstamos e hipotecas (archivo movimientos_financieros_diarios.csv), participando el cálculo por tipo de cuenta para procesar grandes volúmenes en paralelo.

3. **Generación de estados de cuenta anuales:** Compila los movimientos anuales por cuenta (archive estados_financieros_anuales.csv) para producer los estados de cuenta utilizados en auditorías.
4. **Gestión de clientes y cuentas:** Mantenimiento de la información de clientes y ciclo de vida de sus cuentas (apertura, consulta y cierre), expuesto ahora como microservicios independientes.
5. **Procesamiento de pagos y transferencias:** Registro de depósitos, retiros, pagos y transferencias entre cuentas, con verificación de la cuenta de origen/destino y notificación en tiempo real vía eventos Kafka.

Propuesta de Arquitectura

Se propone una arquitectura de microservicios desacoplada, con descubrimiento dinámico de servicios, configuración centralizada, seguridad basada en OAuth2.0 y comunicación híbrida (síncrona vía REST para consultas y validaciones, asíncrona vía Kafka para eventos de negocio). A continuación, se justifican tres requerimientos de negocio que fundamentan las decisiones de diseño.

Requerimiento de negocio 1 — Disponibilidad y tolerancia a fallos

El Banco XYZ no puede permitir que una falla puntual en un servicio (por ejemplo, el microservicio de Cuentas) detenga por completo la atención de pagos o consultas. Por ello, cada llamada entre microservicios de dominio (pagos-service → cuentas-service, cuentas-service → clientes-service) está protegida con Resilience4j (Circuit Breaker, Retry y TimeLimiter), definiendo comportamientos de respaldo (fallback) que permiten degradar el servicio de forma controlada en vez de propagar el fallo a los canales de atención.

Requerimiento de negocio 2 — Experiencia diferenciada por canal

Los tres canales de atención del banco (sucursal/Web, aplicación móvil y cajero automático) tienen necesidades de datos y de seguridad completamente distintas: el canal Web requiere información completa y detallada, el canal móvil requiere respuestas livianas para minimizar el consumo de datos, y el cajero automático requiere únicamente las operaciones mínimas e indispensables (consulta de saldo y retiro) bajo el principio de mínimo privilegio. El patrón BFF permite que cada canal evolucione de forma independiente sin afectar a los demás ni sobrecargar a los microservicios de dominio con lógica de presentación.

Requerimiento de negocio 3 — Escalabilidad ante demanda variable

Las operaciones bancarias presentan picos de demanda (fin de mes, pago de sueldos, fechas de vencimiento), por lo que la plataforma debe poder escalar horizontalmente los microservicios más demandados (Cuentas y Pagos) sin interrumpir el servicio. Al registrar cada instancia en Eureka y balancear la carga mediante Spring Cloud LoadBalancer, es posible levantar múltiples instancias de un mismo microservicio (en Docker Compose mediante --scale, y en AWS mediante Application Auto Scaling sobre ECS) y que el tráfico se distribuya automáticamente entre ellas.

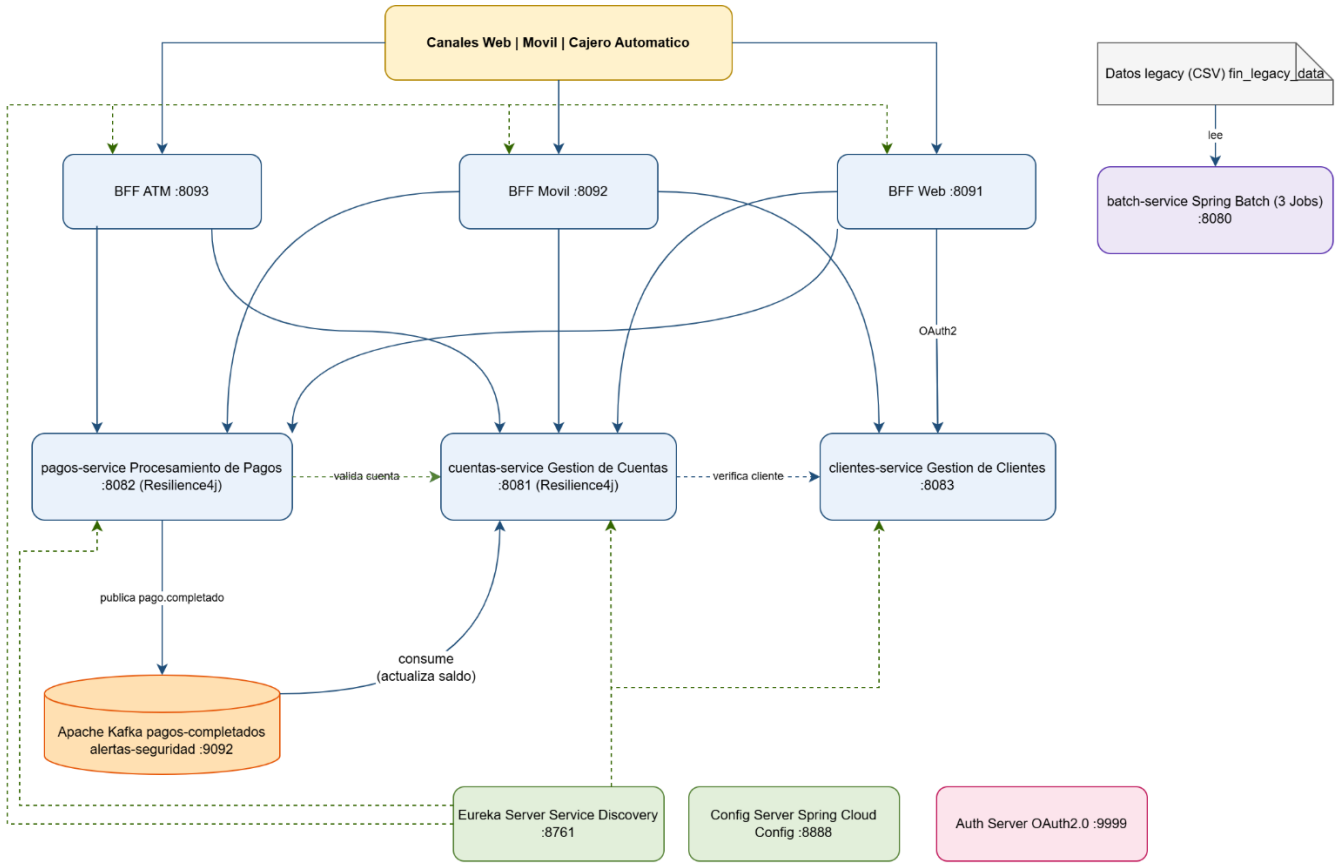


Figura 1. Diagrama de arquitectura general de la solución.

Parte 1: Migración de Procesos Batch con Spring Batch

Los tres procesos batch legacy fueron reescritos como Jobs de Spring Batch dentro del módulo batch-service, leyendo los datasets provistos en el repositorio fin_legacy_data (con datos intencionalmente "sucios": fechas en múltiples formatos, campos vacíos, identificadores duplicados y tipos inválidos) para demostrar un manejo avanzado de errores sobre datos reales de calidad imperfecta, equivalente al escenario de un banco migrando desde mainframe.

Job 1 — Reporte de Transacciones Diarias

Step multi-hilo (SynchronizedItemStreamReader + TaskExecutor con throttleLimit=4) que lee, procesa y escribe los movimientos diarios en chunks de 20 registros, seguido de un

Tasklet que calcula el resumen diario agregado. El procesador intenta parsear la fecha en 4 formatos distintos y, si ninguno aplica (por ejemplo "2024-13-01"), marca el registro como inválido mediante una excepción de tipo skip; además marca como anomalía (sin descartar) los registros con monto vacío o tipo de operación no reconocido.

Job 2 — Cálculo de Intereses Mensuales

Step particionado en 4 particiones paralelas (ahorro, préstamo, hipoteca y "otros"), usando un Partitioner propio y un TaskExecutorPartitionHandler con gridSize=4. Cada partición aplica su propia tasa de interés; los registros con tipo de cuenta corrupto ("-1", "unknown") caen en la partición "otros" y se tratan como anomalía (skip), evidenciando el manejo de calidad de datos legacy sin detener el job completo.

Job 3 — Generación de Estados de Cuenta Anuales

Al igual que el Job 1, utiliza un step multi-hilo para la carga masiva de transacciones anuales (chunk=25, throttleLimit=4), seguido de un Tasklet que compila, por cuenta, los totales de depósitos, retiros y el saldo neto anual mediante una agregación SQL, generando el insumo para las auditorías anuales.

Manejo avanzado de errores, paralelismo y escalabilidad

- **Política de skip:** Los registros no parseables o con datos corruptos se omiten (skipLimit configurado por job) y se registran mediante un listener de skips, sin detener la ejecución del job.
- **Política de retry:** Se simulan fallos transitorios de conexión a base de datos (excepción retryable con retryLimit=3), permitiendo que el job se recupere automáticamente de caídas temporales sin intervención manual.
- **Paralelismo:** Los Jobs 1 y 3 usan lectura/escritura multi-hilo; el Job 2 usa particionamiento real, permitiendo procesar los distintos tipos de cuenta de forma simultánea.
- **Re ejecución:** Cada ejecución se identifica con un parámetro de job único (timestamp), lo que permite re ejecutar cualquier job las veces que sea necesario, por ejemplo, tras un fallo crítico, sin colisionar con ejecuciones previas.

Parte 2: Implementación del Patrón Backend for Frontend (BFF)

Se implementaron tres BFF independientes, cada uno como un microservicio Spring Boot propio, registrado en Eureka y con sus propias credenciales OAuth2.0 de tipo client_credentials, de forma que cada canal solo puede acceder a los scopes que efectivamente necesita.

BFF Web (puerto 8091)

Perfil de acceso: credenciales con todos los scopes (clientes, cuentas y pagos, lectura y escritura). Orientado a navegadores: expone un dashboard que agrega en una sola respuesta los datos del cliente, todas sus cuentas y el historial de pagos de cada una, pensado para interfaces complejas que muestran información completa.

BFF Móvil (puerto 8092)

Perfil de acceso: scopes cuentas.read, pagos.read, pagos.write y clientes.read. Orientado a aplicaciones móviles: expone un endpoint de resumen liviano (nombre del cliente, saldo total y listado resumido de cuentas) que minimiza el tamaño de la respuesta y el consumo de ancho de banda frente al canal Web.

BFF Cajero Automático (puerto 8093)

Perfil de acceso: únicamente cuentas.read y pagos.write (mínimo privilegio).
Orientado a operaciones críticas y acotadas: solo expone consulta de saldo y retiro de efectivo, sin acceso a ningún otro dato del cliente, reduciendo al mínimo la superficie de exposición de este canal, que es el más expuesto físicamente.
La autenticación y autorización de cada canal se gestiona mediante tokens JWT emitidos por el servidor OAuth2.0 (auth-server), validados contra el JWKS en cada microservicio de dominio. Todas las comunicaciones ocurren sobre HTTP con cabecera Authorization: Bearer, y en el despliegue en AWS se exponen únicamente a través del Application Load Balancer con certificado TLS (ver despliegue.md).

Parte 3: Desarrollo de Microservicios Resilientes con Spring Cloud

Los tres microservicios de dominio — Gestión de Clientes, Gestión de Cuentas y Procesamiento de Pagos — se construyeron de forma independiente, cada uno con su propia base de datos (H2 en el entorno académico, reemplazable por RDS en producción), y se integran mediante Spring Cloud Config, Eureka, OAuth2.0, Resilience4j y Kafka.

Gestión de configuración y descubrimiento

- **Spring Cloud Config Server:** centraliza la configuración de todos los microservicios mediante un perfil nativo, exponiendo propiedades comunes (actuator, resiliencia) desde un único punto.
- **Eureka Server:** permite el registro y descubrimiento dinámico de instancias; las llamadas entre servicios usan URIs lb://, resueltas por Spring Cloud LoadBalancer sin direcciones fijas.

Seguridad con OAuth2.0

El auth-server actúa como servidor de autorización (grant client_credentials) con seis clientes registrados (uno por microservicio de dominio y uno por BFF), cada uno con scopes acotados a su función. Cada microservicio de dominio actúa como resource server, validando el JWT contra el JWKS del auth-server y exigiendo el scope correspondiente (lectura o escritura) en cada endpoint.

Resiliencia con Resilience4j

Las llamadas pagos-service → cuentas-service y cuentas-service → clientes-service están envueltas con @CircuitBreaker, @Retry y @TimeLimiter. Ante fallos repetidos, el circuito pasa a estado OPEN y las siguientes llamadas se resuelven de inmediato contra un método de fallback (sin intentar la red), evitando que la indisponibilidad de un microservicio se propague en cascada; al recuperarse el servicio, el circuito transita automáticamente a HALF_OPEN y luego a CLOSED.

Eventos en tiempo real con Kafka

Cada pago registrado en pagos-service publica un evento en el tópico pagoscompletados, consumido de forma asíncrona por cuentas-service para actualizar el saldo correspondiente. Adicionalmente, los pagos sobre un monto umbral publican una alerta en el tópico alertas-seguridad, simulando la detección de operaciones de mayor riesgo. Productor y consumidor usan DTOs independientes y desactivan las cabeceras de tipo Java, desacoplando completamente ambos módulos.

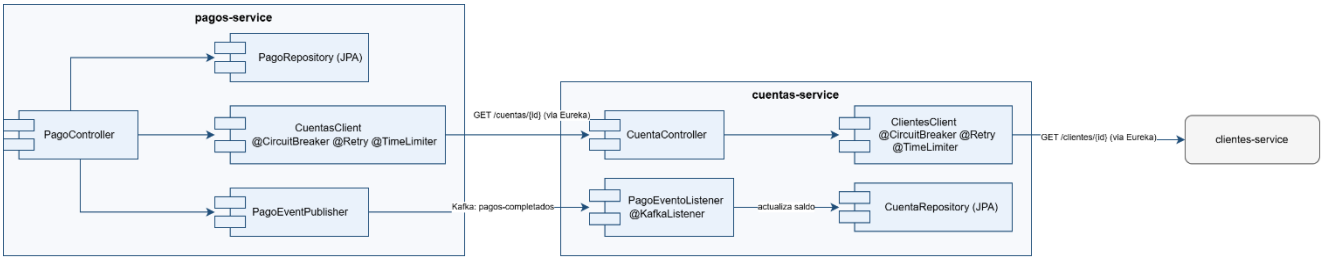


Figura 2. Diagrama de componentes de pago-service y cuentas-service.

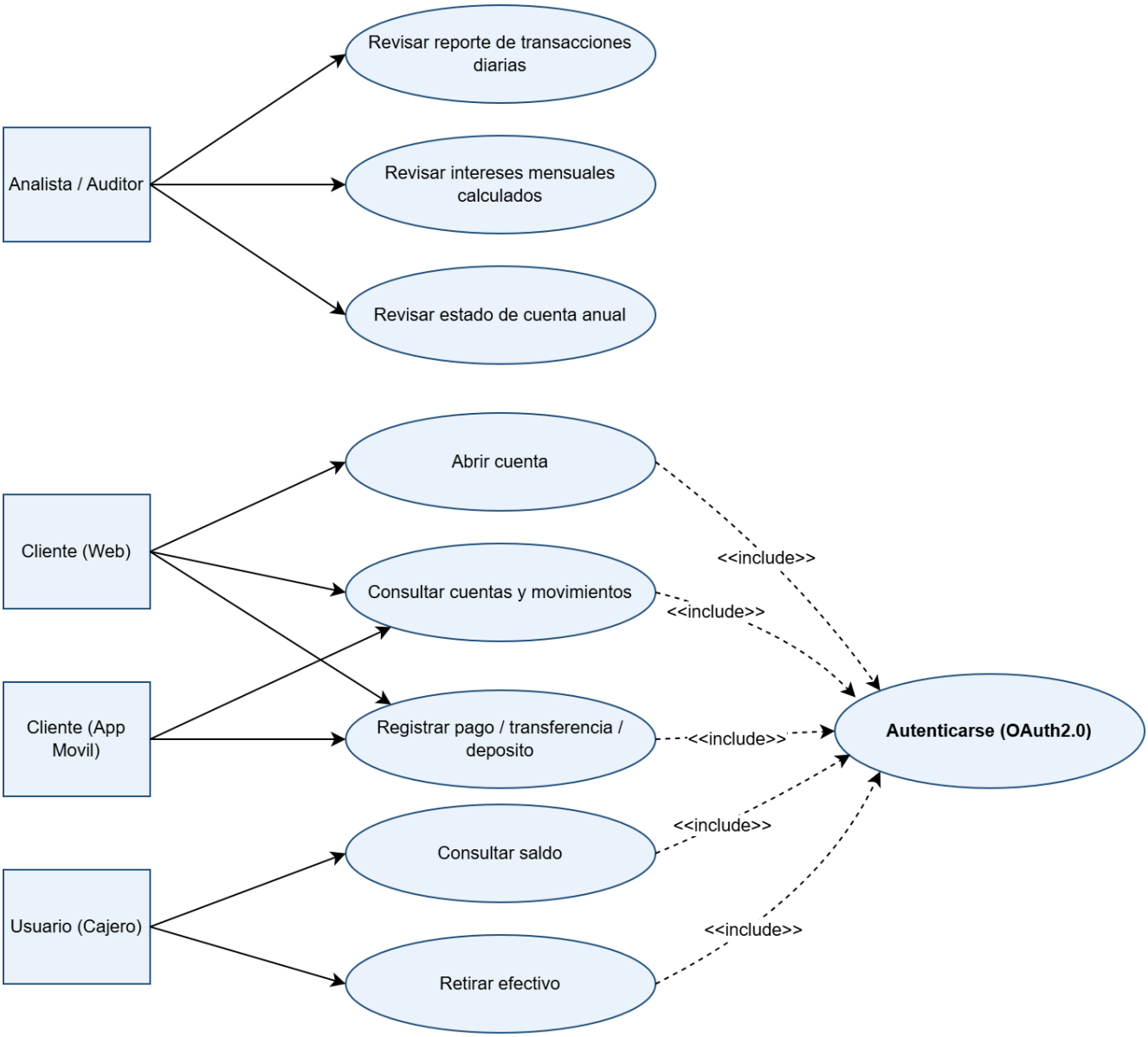


Figura 3. Diagrama de casos de uso de la plataforma.

Docker y Escalabilidad Horizontal

Los diez módulos de la solución se contenedorizan mediante Dockerfiles multi-stage (build con Maven sobre Eclipse Temurin 21, ejecución sobre una imagen JRE Alpine liviana) y se orquestan en conjunto mediante un docker-compose.yaml que incluye además un contenedor de Apache Kafka en modo KRaft.

Diseño para escalar: los tres microservicios de dominio (clientes-service, cuentasservice y pagos-service) se definieron sin puerto fijo publicado hacia el host, exponiendo el puerto solo internamente (expose), precisamente para permitir levantar múltiples instancias simultáneas con docker compose up -d --scale sin conflictos de puerto.

Al escalar, por ejemplo, cuentas-service a 3 instancias, cada una se registra en Eureka de forma independiente con su propia IP interna; las llamadas desde los BFF y desde pagosservice, al usar lb://cuentas-service, se distribuyen automáticamente entre las instancias disponibles mediante Spring Cloud LoadBalancer, sin reiniciar ningún otro componente del sistema. Este mismo mecanismo se traslada al despliegue en AWS mediante Application Auto Scaling sobre ECS Fargate, como se detalla en despliegue.md.

Despliegue en la nube (AWS)

La estrategia de despliegue objetivo es Amazon ECS con Fargate: las imágenes Docker ya construidas se publican en ECR, la red interna se resuelve mediante AWS Cloud Map, las credenciales OAuth2.0 se gestionan en Secrets Manager, las bases H2 se reemplazan por Amazon RDS y el contenedor único de Kafka se reemplaza por un clúster Amazon MSK. Cada microservicio de dominio se despliega con un desired-count inicial de 3 instancias y una política de Application Auto Scaling basada en CPU, mientras que los tres BFF quedan expuestos públicamente a través de un Application Load Balancer con TLS. El job de Spring Batch se ejecuta como una ECS Scheduled Task mediante EventBridge, evitando mantenerlo activo 24/7. El detalle completo de comandos y configuraciones se encuentra en el archivo despliegue.md que acompaña esta entrega.

Documentación del Proyecto

Junto con el presente informe técnico, la entrega incluye los siguientes documentos y componentes, todos ubicados en la misma carpeta del proyecto:

- **readme.md:** Enlace al repositorio GitHub del código fuente y descripción de la organización del proyecto (10 módulos Maven).
- **instrucciones.md:** Guía paso a paso para compilar, ejecutar y probar cada componente del sistema, tanto de forma local como mediante Docker Compose.

- **despliegue.md:** procedimiento detallado para desplegar la solución completa en AWS.
- **Código fuente completo:** los 10 módulos Maven (config-server, eureka-server, auth-server, clientes-service, cuentas-service, pagos-service, bff-web, bff-mobile, bff-atm y batch-service), con sus respectivos Dockerfiles y el docker-compose.yaml de orquestación.

Conclusiones

El proyecto desarrollado demuestra una migración integral del sistema legacy del Banco XYZ hacia una arquitectura de microservicios moderna: los procesos batch críticos ahora se ejecutan sobre Spring Batch con manejo avanzado de errores, reintentos y paralelismo real; los tres canales de atención cuentan con un BFF propio que optimiza la experiencia y la seguridad de cada uno; y los microservicios de dominio operan de forma resiliente, segura y escalable, apoyados en Spring Cloud, OAuth2.0, Resilience4j y Kafka. La plataforma completa está lista para escalar horizontalmente, tanto en el entorno local de pruebas como en un despliegue productivo en AWS, cumpliendo así los objetivos de flexibilidad, resiliencia y capacidad de adaptación planteados en el caso del Banco XYZ.

Reservados todos los derechos Fundación Instituto Profesional Duoc UC. No se permite copiar, reproducir, reeditar, descargar, publicar, emitir, difundir, de forma total o parcial la presente obra, ni su incorporación a un sistema informático, ni su transmisión en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopia, grabación u otros) sin autorización previa y por escrito de Fundación Instituto Profesional Duoc UC
La infracción de dichos derechos puede constituir un delito contra la propiedad intelectual.